Skill – Week12

Session 23:

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
  GNU nano 8.7.1                          pipeline.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    long long sum = 0;

    for (long long i = 1; i <= 1000000; i++) {
        sum += i;
    }

    printf("%lld\n", sum);

    return 0;
}
```

```
manikanta@Hemu:~/OS_Lab$ mkdir pipeline_lab
manikanta@Hemu:~/OS_Lab$ cd pipeline_lab
manikanta@Hemu:~/OS_Lab/pipeline_lab$ nano pipeline.c
manikanta@Hemu:~/OS_Lab/pipeline_lab$ gcc -O2 -Wall -Wextra pipeline.c -o
 pipeline
manikanta@Hemu:~/OS_Lab/pipeline_lab$ ./pipeline
500000500000
manikanta@Hemu:~/OS_Lab/pipeline_lab$ ./pipeline | tee output.txt | sort
| uniq -c
      1 500000500000
manikanta@Hemu:~/OS_Lab/pipeline_lab$ time ./pipeline | tee output.txt |
sort | uniq -c
      1 500000500000

real    0m0.004s
user    0m0.005s
sys     0m0.004s
manikanta@Hemu:~/OS_Lab/pipeline_lab$ ./pipeline > job1.txt &
./pipeline > job2.txt &
./pipeline > job3.txt &
./pipeline > job4.txt &
[1] 1396
[2] 1402
[1]   Done                    ./pipeline > job1.txt
[3] 1408
[2]   Done                    ./pipeline > job2.txt
[4] 1414
[3]   Done                    ./pipeline > job3.txt
[4]+  Done                    ./pipeline > job4.txt
```

Report: Large pipelines were executed, and multiple jobs were launched simultaneously using background processes. System stability and resource usage were monitored using top, free -h, and ps. Job failures were checked using exit statuses, and execution results were recorded in report.csv for performance analysis.

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

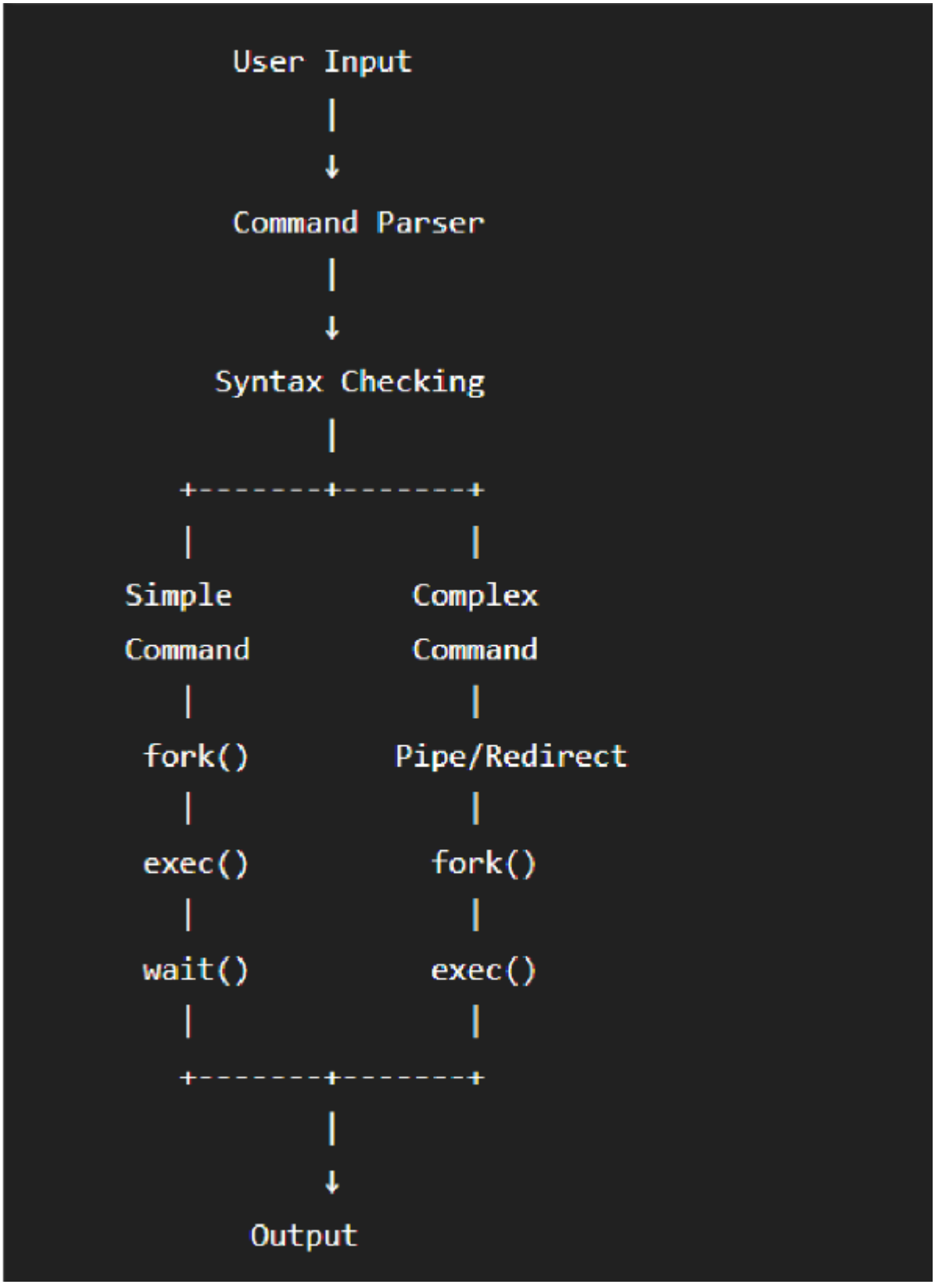
```
  GNU nano 8.7.1                                main.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
    } else {
        wait(NULL);
        printf("Parent process: PID = %d\n", getpid());
    }

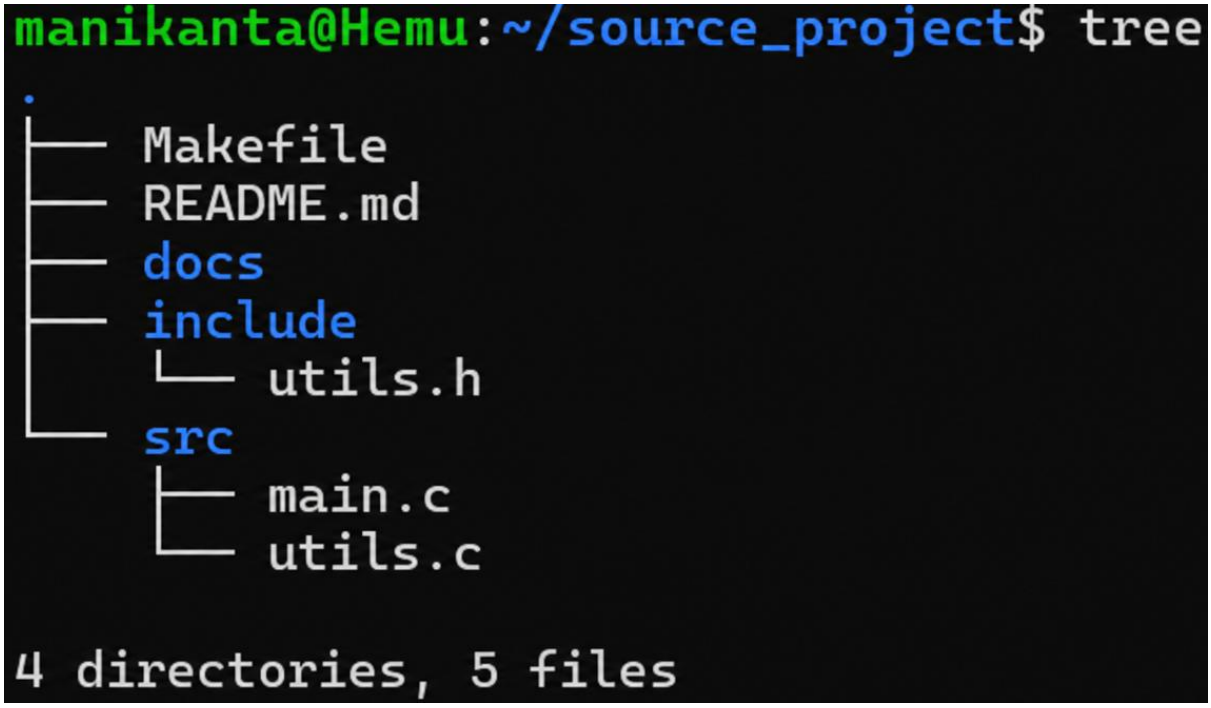
    return 0;
}
```



Report: The system architecture was documented using README.md, and the execution flow was represented using a Graphviz diagram. System calls such as fork(), wait(), and getpid() were explained.

Session 24:

1. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables



```
 GNU nano 8.7.1                    src/utils.c *
#include <stdio.h>
#include "utils.h"

void display_message(void) {
    printf("Project build successful!\n");
}
```

```
 GNU nano 8.7.1                    README.md *
# Source Code Organization Project

## Description
This project demonstrates modular C programming,
build automation, and release management using Git.

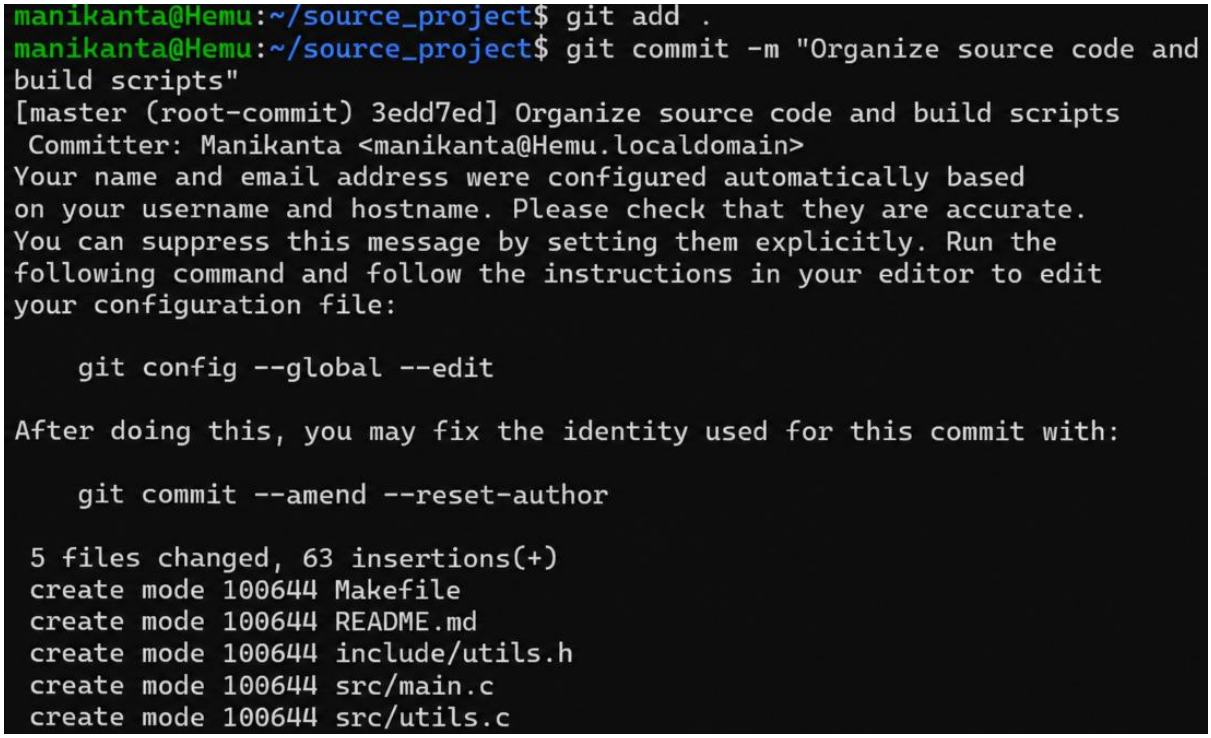
## Project Structure
- src/: Contains C source files.
- include/: Contains header files.
- docs/: Contains project documentation.
- Makefile: Automates compilation and cleanup.
- README.md: Project description and instructions.

## Compilation
Run:
make

## Execution
Run:
make run

## Cleanup
Run:
make clean

## Tools Used
GCC, GNU Make, Git, and Ubuntu Linux.
```



Report: The source code was organized into modular directories, and compilation was automated using a Makefile. Project documentation was added using README.md, repository structure was verified using tree, and Git was used to commit the project and tag version v1.0. The final build and execution were validated using make, ./app, and git status.

2. To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
  GNU nano 8.7.1                                demo.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_signal(int sig) {
    printf("\nSignal received: %d\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);

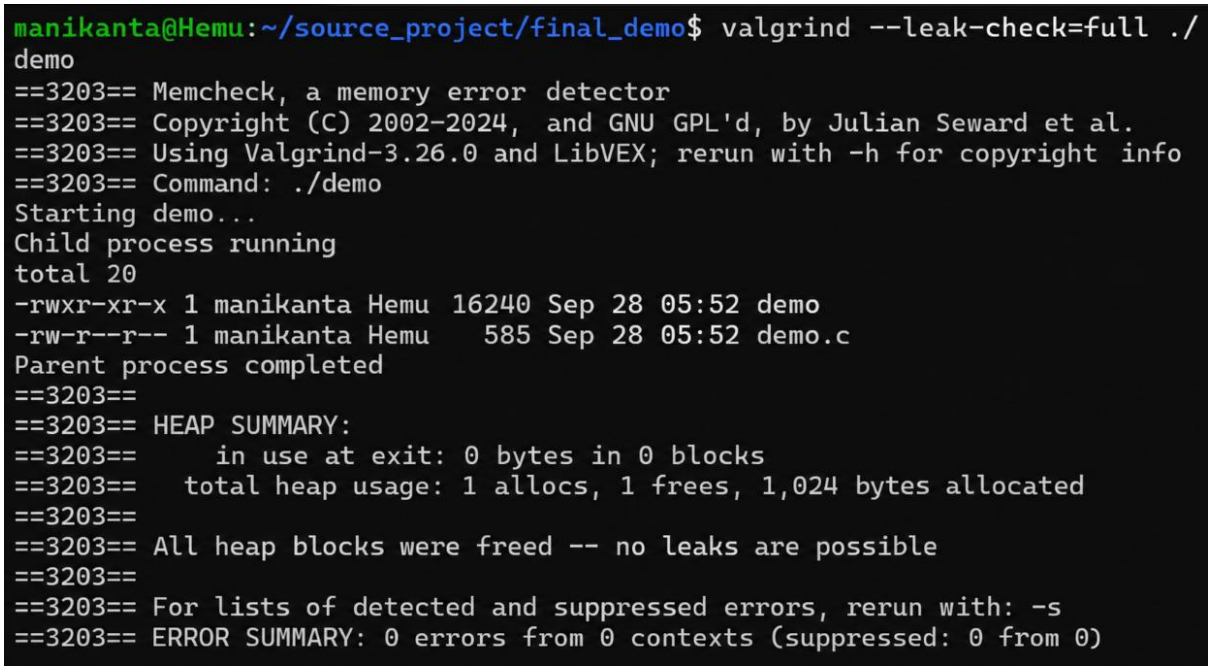
    printf("Starting demo...\n");

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process running\n");
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        return 1;
    } else if (pid > 0) {
        wait(NULL);
        printf("Parent process completed\n");
    } else {
        perror("fork");
        return 1;
    }

    return 0;
}
```

```
manikanta@Hemu:~/source_project/final_demo$ nano demo.c
manikanta@Hemu:~/source_project/final_demo$ gcc -Wall -Wextra demo.c -o d
emo
manikanta@Hemu:~/source_project/final_demo$ ./demo
Starting demo...
Child process running
total 20
-rwxr-xr-x 1 manikanta Hemu 16240 Sep 28 05:52 demo
-rw-r--r-- 1 manikanta Hemu   585 Sep 28 05:52 demo.c
Parent process completed
```



Report: The project was successfully tested by demonstrating pipelines, process signals, and resource monitoring. All major features were validated, and the final presentation was reviewed.